

Correctness of Time-forward Processing Algorithms

Steffan Sølvsten, Simon Wimmer

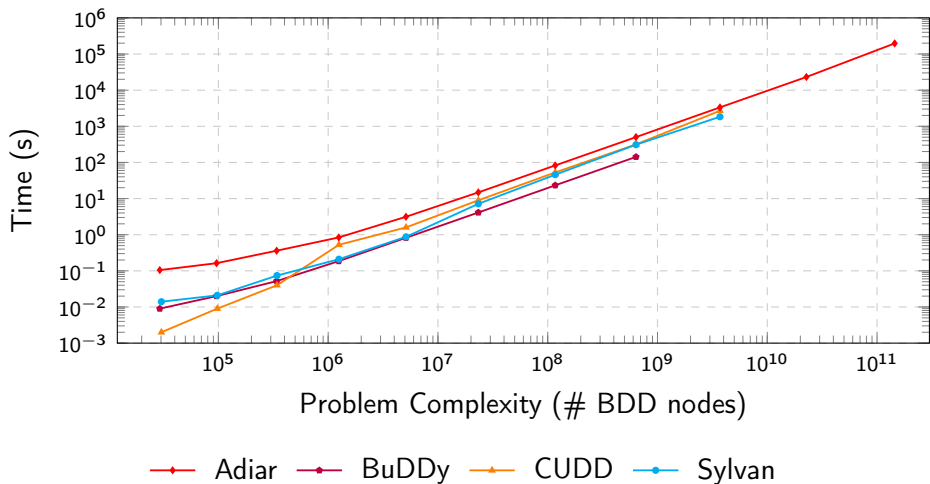
LogSem Seminar, 2nd of December 2024



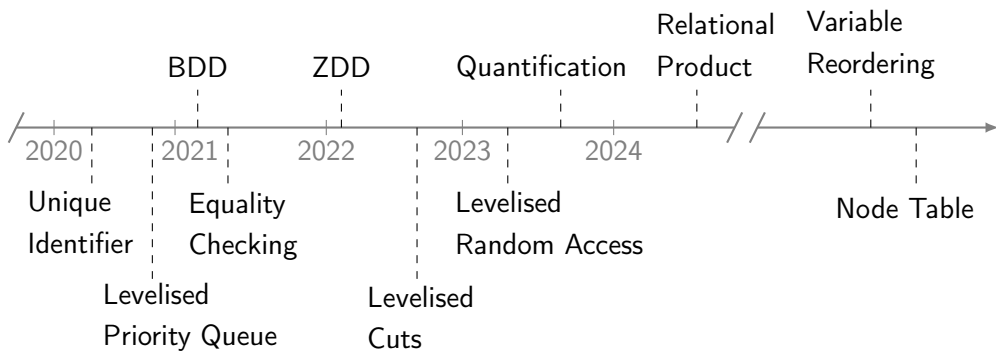
Adiar

I/O-efficient Decision Diagrams

github.com/logsem/adiar



Running Time to solve N -Queens problems.



Written in

C++

```
////////////////////////////////////  
/// \brief Negates the content of 'p' if it is a terminal and the  
///      'negate' flag is set to true.  
////////////////////////////////////
```

```
1  inline ptr_uint64  
2  cnot(const ptr_uint64& p, const bool negate)  
3  {  
4      const uint64 shifted_negate =  
5          ((uint64) negate) << ptr_uint64::data_shift;  
6      return p.is_leaf() ? p._raw ^ shifted_negate : p._raw;  
7  }
```

Correctness guaranteed by

~3500 Unit Tests

~400 Integration Tests

Cache and I/O Efficient Functional Algorithms

Guy E. Blelloch Robert Harper

Carnegie Mellon University

Abstract

In this paper we present a cost model for analyzing the memory efficiency of algorithms expressed in a simple functional language. We show how some algorithms written in standard forms using just lists and trees (no arrays) and requiring no explicit memory layout or memory management are efficient in the model. We then describe an implementation of the language and show provable bounds for mapping the cost in our model to the cost in the ideal-cache model. **These bound imply that purely functional programs based on lists and trees with no special attention to any details of memory layout can be as asymptotically as efficient as the carefully designed imperative I/O efficient algorithms.** For example we describe an $\mathcal{O}(N/B \log_{M/B} N/B)$ cost sorting algorithm, which is optimal in the ideal cache and I/O models.

Contents

Motivation

Correctness of Time-forward Processing

Encoding Binary Decision Diagrams

`bbd_eval`

`bbd_not`

`bbd_satcount`

Appendix

$\lfloor a \rfloor_n$: Bounded Domain

$\#_n$: Number of Assignments

Contents

Motivation

Correctness of Time-forward Processing

Encoding Binary Decision Diagrams

`bdd_eval`

`bdd_not`

`bdd_satcount`

Appendix

$[a]_n$: Bounded Domain

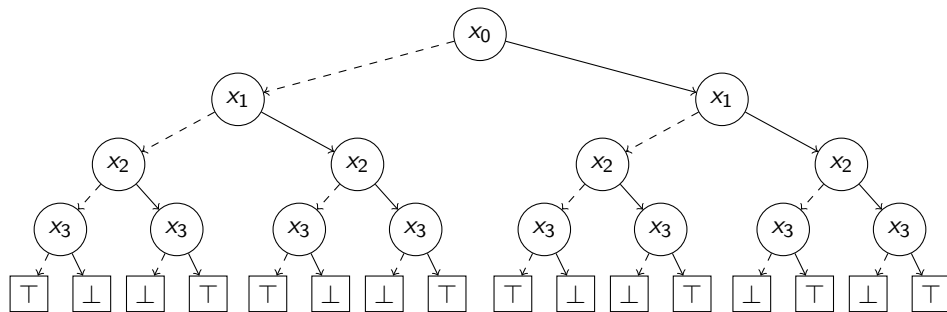
$\#_n$: Number of Assignments

Semantics of BDDs

$$f(x_0, x_1, x_2, x_3) \equiv (x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$$

¹Julius Michaelis, Maximilian Haslbeck, Peter Lammich, and Lars Hupel. “Algorithms for Reduced Ordered Binary Decision Diagrams”. In: Archive of Formal Proofs (2016)

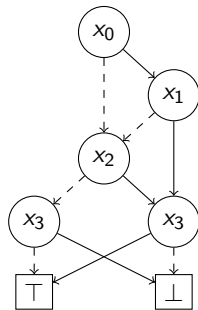
Semantics of BDDs



Binary Decision Tree

¹Julius Michaelis, Maximilian Haslbeck, Peter Lammich, and Lars Hupel. “Algorithms for Reduced Ordered Binary Decision Diagrams”. In: Archive of Formal Proofs (2016)

Semantics of BDDs



Binary Decision Diagram

¹Julius Michaelis, Maximilian Haslbeck, Peter Lammich, and Lars Hupel. “Algorithms for Reduced Ordered Binary Decision Diagrams”. In: *Archive of Formal Proofs* (2016)

Data Types: Unique Identifiers and Pointers

```
1 Uid = (level:  $\mathbb{N}$ , id:  $\mathbb{N}$ )

2 operator < (a: Uid) (b: Uid) =
3     a.level < b.level  $\vee$  (a.level = b.level  $\wedge$  a.id < b.id)

4 Ptr = Leaf (val :  $\mathbb{B}$ )
5     | Node (uid : Uid)

5 operator < (a: Ptr) (b: Ptr) =
6     lift Uid.< s.t. Ptr.Node < Ptr.Leaf
```

Lemma

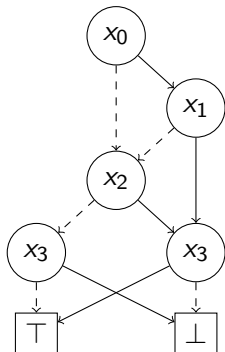
Uid.< and Ptr.< are total orders.

Proof.

Trivial case distinctions.



Data Types: Nodes and BDDs

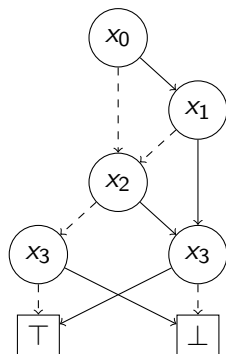


$$(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$$

```
1 Node = (i: Uid, t: Ptr, e: Ptr)
2 Bdd  = Leaf  (val :  $\mathbb{B}$ )
3      | Nodes (ns : List[Node])

4 Example = Bdd.Nodes([
5   Node(Uid(0,0), Ptr(1,0), Ptr(2,0));
6   Node(Uid(1,0), Ptr(3,1), Ptr(2,0));
7   Node(Uid(2,0), Ptr(3,1), Ptr(3,0));
8   Node(Uid(3,0), Ptr( $\perp$ ),  Ptr( $\top$ ));
9   Node(Uid(3,1), Ptr( $\top$ ),  Ptr( $\perp$ ));
10 ])
```

Data Types: Nodes and BDDs



$$(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$$

```
1 Node = (i: Uid, t: Ptr, e: Ptr)
2 Bdd  = Leaf  (val :  $\mathbb{B}$ )
3       | Nodes (ns : List[Node])
```

Definition

A Bdd is *well formed*, if $ns : \text{List}[\text{Node}]$ satisfies:

- 1 It is *non-empty*.
- 2 It is *closed*, i.e. every node referred to exists.
- 3 For each node, the level is strictly increasing.
- 4 It is *sorted* w.r.t. Node.i .

Contents

Motivation

Correctness of Time-forward Processing

Encoding Binary Decision Diagrams

`bbd_eval`

`bbd_not`

`bbd_satcount`

Appendix

$[a]_n$: Bounded Domain

$\#_n$: Number of Assignments

`bdd_eval f x`

```
1 bdd_eval' ((i t e)::ns': List[Node]) (tgt: Uid) (x:  $\mathbb{N} \rightarrow \mathbb{B}$ ) =  
2   if i < tgt  
3   then bdd_eval' ns' tgt x  
4   else case if x i.level then t else e of  
5       Leaf(b)       $\rightarrow$  b  
6       Node(tgt')  $\rightarrow$  bdd_eval' ns' tgt' x  
  
7 bdd_eval (val      : Bdd.Leaf)  (x:  $\mathbb{N} \rightarrow \mathbb{B}$ ) = val  
8 bdd_eval (r::ns   : Bdd.Nodes) (x:  $\mathbb{N} \rightarrow \mathbb{B}$ ) = bdd_eval' r::ns r x
```

`bdd_eval f x`

Define the function `bdt_of_bdd : Bdd  $\rightarrow$  Bdt` to convert a Binary Decision Diagram into a Binary Decision Tree. Here, skip over “irrelevant” nodes to convert subtrees.

bdd_eval f x

Define the function $\text{bdt_of_bdd} : \text{Bdd} \rightarrow \text{Bdt}$ to convert a Binary Decision Diagram into a Binary Decision Tree. Here, skip over “irrelevant” nodes to convert subtrees.

Theorem

If f is well formed, then $\forall x: \text{bdd_eval } f \ x \iff \text{bdt_eval } (\text{bdt_of_bdd } f) \ x$.

Proof.

Case Leaf $\mathbb{B}$: Trivial

Case Nodes ns :

Induction on ns .

Discard bad cases due to the BDD being *closed* and *sorted*.



Contents

Motivation

Correctness of Time-forward Processing

Encoding Binary Decision Diagrams

`bbd_eval`

`bbd_not`

`bbd_satcount`

Appendix

$[a]_n$: Bounded Domain

$\#_n$: Number of Assignments

`bdd_not f`

```
1 operator ! (p: Ptr) = case p of Leaf v → Leaf !v
2                               Node u → Node u

4 bdd_not (val : Bdd.Leaf)  = Bdd.Leaf !v
5 bdd_not (ns   : Bdd.Nodes) = Bdd.Nodes (map λ(i t e) → (i !t !e) ns)
```


bdd_not f

```
1 operator ! (p: Ptr) = case p of Leaf v → Leaf !v
2                               Node u → Node u

4 bdd_not (val : Bdd.Leaf)  = Bdd.Leaf !v
5 bdd_not (ns   : Bdd.Nodes) = Bdd.Nodes (map λ(i t e) → (i !t !e) ns)
```

Theorem

If f is well formed, then $\forall x: \neg(\text{bdd_eval } f \ x) \iff \text{bdd_eval } (\text{bdd_not } f) \ x$.

Proof.

Case Leaf $\mathbb{B}$: Trivial

Case Nodes ns: Induction on ns.



Contents

Motivation

Correctness of Time-forward Processing

Encoding Binary Decision Diagrams

`bbd_eval`

`bbd_not`

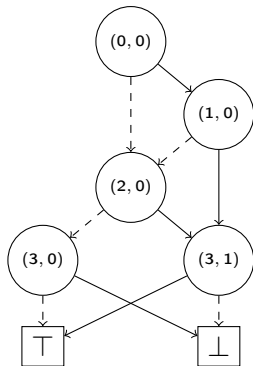
`bbd_satcount`

Appendix

$[a]_n$: Bounded Domain

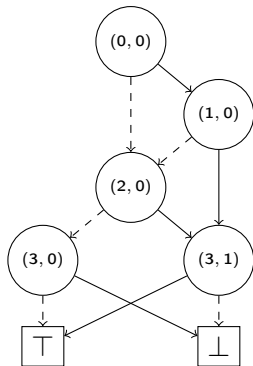
$\#_n$: Number of Assignments

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

bdd_pathcount f



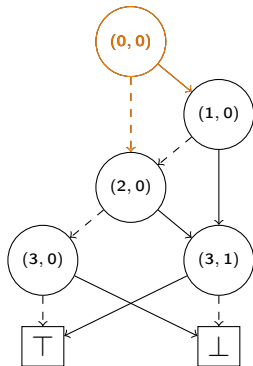
(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

Priority Queue: Q_{count} :

[

]

bdd_pathcount f



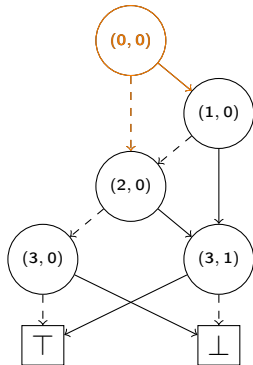
(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

Priority Queue: Q_{count} :

[

]

bdd_pathcount f



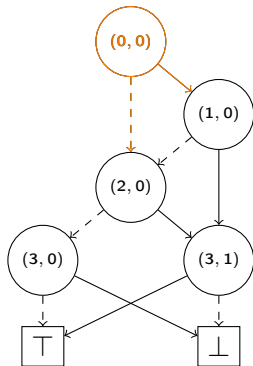
(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

Priority Queue: Q_{count} :

[$((0,0) \xrightarrow{T} (1,0), 1)$,
 $((0,0) \xrightarrow{\perp} (2,0), 1)$,

]

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

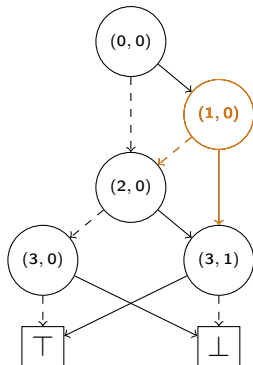
Seek	Sum	Result
(1, 0)	0	0

Priority Queue: Q_{count} :

[	$((0,0) \xrightarrow{\top} (1,0), 1)$	,
	$((0,0) \xrightarrow{\perp} (2,0), 1)$	,

]

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

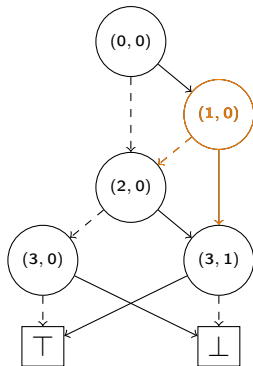
Seek	Sum	Result
(1, 0)	0	0

Priority Queue: Q_{count} :

[$((0, 0) \xrightarrow{T} (1, 0), 1)$,
 $((0, 0) \xrightarrow{\perp} (2, 0), 1)$,

]

bdd_pathcount f



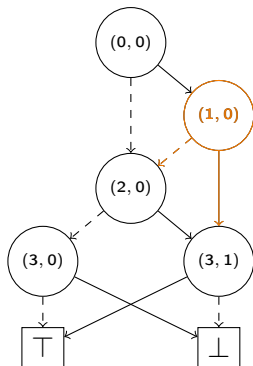
(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

Seek	Sum	Result
(1, 0)	1	0

Priority Queue: Q_{count} :

[
 $((0, 0) \xrightarrow{\perp} (2, 0), 1)$,
]

bdd_pathcount f



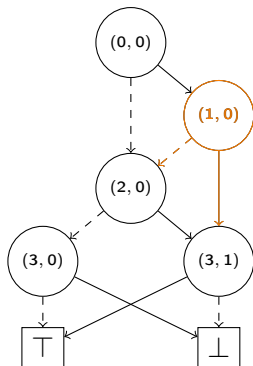
(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

Seek	Sum	Result
(1, 0)	1	0

Priority Queue: Q_{count} :

[	
$((0, 0) \xrightarrow{\perp} (2, 0), 1)$	,
$((1, 0) \xrightarrow{\perp} (2, 0), 1)$	,
$((1, 0) \xrightarrow{\top} (3, 1), 1)$	,
]	

bdd_pathcount f



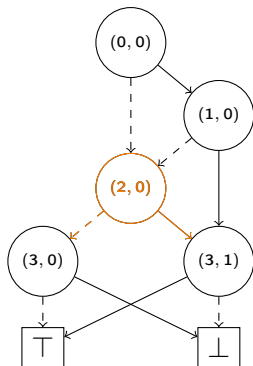
(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

Seek	Sum	Result
(2, 0)	0	0

Priority Queue: Q_{count} :

[	
$((0, 0) \xrightarrow{\perp} (2, 0), 1)$	,
$((1, 0) \xrightarrow{\perp} (2, 0), 1)$	,
$((1, 0) \xrightarrow{\top} (3, 1), 1)$	,
]	

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

Seek	Sum	Result
(2, 0)	0	0

Priority Queue: Q_{count} :

[

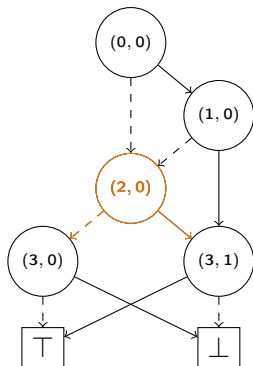
$((0, 0) \xrightarrow{\perp} (2, 0), 1)$,

$((1, 0) \xrightarrow{\perp} (2, 0), 1)$,

$((1, 0) \xrightarrow{\top} (3, 1), 1)$,

]

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

Seek	Sum	Result
(2, 0)	1	0

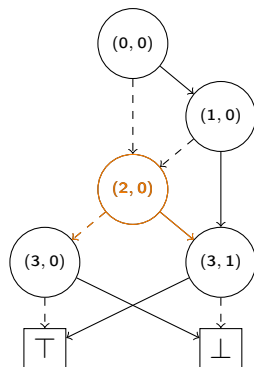
Priority Queue: Q_{count} :

[$((1, 0) \xrightarrow{\perp} (2, 0), 1)$,

$((1, 0) \xrightarrow{\top} (3, 1), 1)$,

]

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

Seek	Sum	Result
(2, 0)	2	0

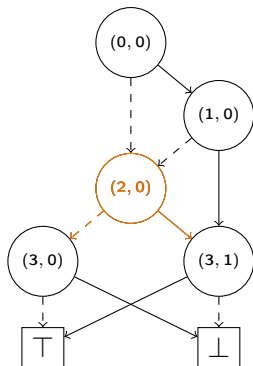
Priority Queue: Q_{count} :

[

$((1, 0) \xrightarrow{T} (3, 1), 1)$,

]

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

Seek	Sum	Result
(2, 0)	2	0

Priority Queue: Q_{count} :

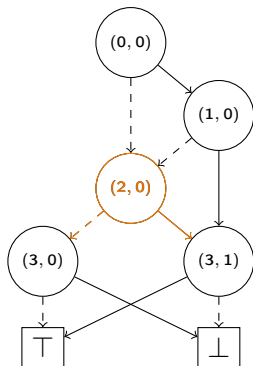
[

$$((2, 0) \xrightarrow{\perp} (3, 0), \quad 2) \quad ,$$

$$((1, 0) \xrightarrow{\top} (3, 1), \quad 1) \quad ,$$

$$((2, 0) \xrightarrow{\top} (3, 1), \quad 2) \quad]$$

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

Seek	Sum	Result
(3, 0)	0	0

Priority Queue: Q_{count} :

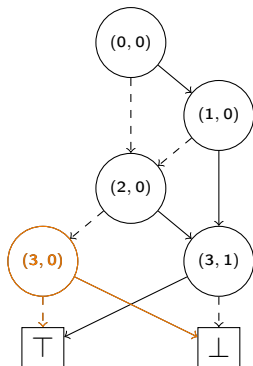
[

$$((2, 0) \xrightarrow{\perp} (3, 0), \quad 2) \quad ,$$

$$((1, 0) \xrightarrow{\top} (3, 1), \quad 1) \quad ,$$

$$((2, 0) \xrightarrow{\top} (3, 1), \quad 2) \quad]$$

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

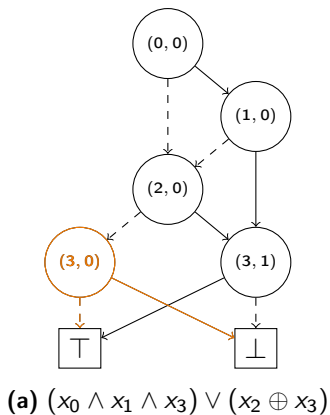
Seek	Sum	Result
(3, 0)	0	0

Priority Queue: Q_{count} :

[

$((2, 0) \xrightarrow{\perp} (3, 0), 2)$,
 $((1, 0) \xrightarrow{\top} (3, 1), 1)$,
 $((2, 0) \xrightarrow{\top} (3, 1), 2)$]

bdd_pathcount f



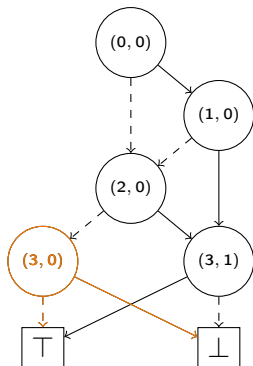
Seek	Sum	Result
(3, 0)	2	0

Priority Queue: Q_{count} :

[

$((1, 0) \xrightarrow{\top} (3, 1), 1)$,
 $((2, 0) \xrightarrow{\top} (3, 1), 2)$]

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

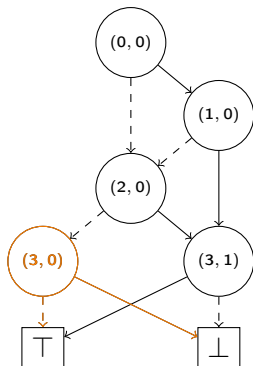
Seek	Sum	Result
(3, 0)	2	2

Priority Queue: Q_{count} :

[

$((1, 0) \xrightarrow{T} (3, 1), 1)$,
 $((2, 0) \xrightarrow{T} (3, 1), 2)$]

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

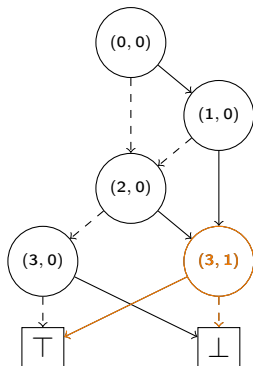
Seek	Sum	Result
(3, 1)	0	2

Priority Queue: Q_{count} :

[

$((1, 0) \xrightarrow{T} (3, 1), 1)$,
 $((2, 0) \xrightarrow{T} (3, 1), 2)$]

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

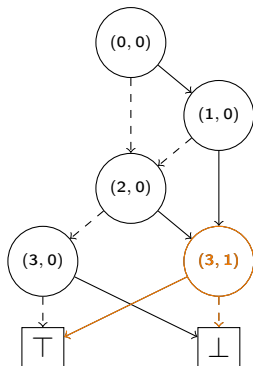
Seek	Sum	Result
(3, 1)	0	2

Priority Queue: Q_{count} :

[

$((1, 0) \xrightarrow{T} (3, 1), 1)$,
 $((2, 0) \xrightarrow{T} (3, 1), 2)$]

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

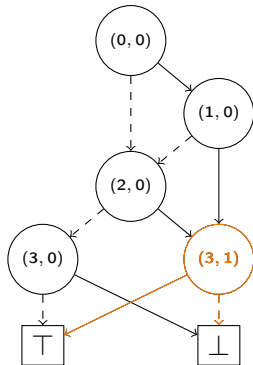
Seek	Sum	Result
(3, 1)	1	2

Priority Queue: Q_{count} :

[

$((2, 0) \xrightarrow{T} (3, 1), \quad 2) \quad]$

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

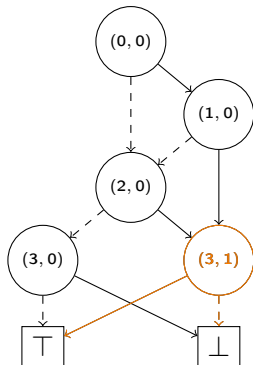
Seek	Sum	Result
(3, 1)	3	2

Priority Queue: Q_{count} :

[

]

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

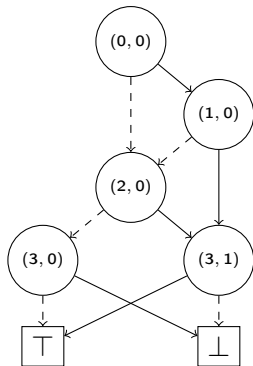
Seek	Sum	Result
(3, 1)	3	5

Priority Queue: Q_{count} :

[

]

bdd_pathcount f



(a) $(x_0 \wedge x_1 \wedge x_3) \vee (x_2 \oplus x_3)$

Result

5

Priority Queue: Q_{count} :

[

]

`bdd_satcount f vc`

What needs to be changed for a
`bdd_satcount f vc`?

bdd_satcount f vc

Think of a Priority Queue of Req as a *Multiset* with (pure) functions top() and pop().

```
1 Req = (target : Uid, sum :  $\mathbb{N}$ , levels_visited :  $\mathbb{N}$ )  
  
2 operator (a: Req) < (b: Req) = a.target < b.target  
3    $\vee$  (a.target = b.target  $\wedge$  a.levels_visited < b.levels_visited)
```

Lemma

Req.< is a partial order.

Proof.

Trivial case distinctions.



bdd_satcount f vc

Think of a Priority Queue of Req as a *Multiset* with (pure) functions `top()` and `pop()`.

```
1 Req = (target : Uid, sum : ℕ, levels_visited : ℕ)
```

Definition

$$\#^{\text{Req}} ns (\text{Req } t \ s \ \ell) \triangleq s \cdot (\#_{\ell} ns \ t) \quad \text{and} \quad \#^{\text{Pq}} ns \ pq \triangleq \sum_{r \in pq} \#^{\text{Req}} ns \ r.$$

bdd_satcount f vc

Think of a Priority Queue of Req as a *Multiset* with (pure) functions `top()` and `pop()`.

```
1 Req = (target : Uid, sum : ℕ, levels_visited : ℕ)
```

Definition

$$\#^{\text{Req}} ns (\text{Req } t \ s \ \ell) \triangleq s \cdot (\#_{\ell} ns \ t) \quad \text{and} \quad \#^{\text{pq}} ns \ pq \triangleq \sum_{r \in pq} \#^{\text{Req}} ns \ r.$$

Lemma

$$\#^{\text{pq}} ns \ \emptyset = 0$$

Proof.

Trivial. □

bdd_satcount f vc

Think of a Priority Queue of Req as a *Multiset* with (pure) functions `top()` and `pop()`.

```
1 Req = (target : Uid, sum : ℕ, levels_visited : ℕ)
```

Definition

$$\#^{\text{Req}} ns (\text{Req } t \ s \ \ell) \triangleq s \cdot (\#_{\ell} ns \ t) \quad \text{and} \quad \#^{\text{pq}} ns \ pq \triangleq \sum_{r \in pq} \#^{\text{Req}} ns \ r.$$

Lemma

If $pq.\text{top}() = \text{Some } r$, then $\#^{\text{pq}} ns \ pq = \#^{\text{pq}} ns \ pq.\text{pop}() + \#^{\text{Req}} ns \ r$

Proof.

Due to $pq = \{r\} + pq.\text{pop}()$. □

bdd_satcount f vc

Think of a Priority Queue of Req as a *Multiset* with (pure) functions top() and pop().

1 Req = (target : Uid, sum : $\mathbb{N}$, levels_visited : $\mathbb{N}$)

Definition

$$\#^{\text{Req}} ns (\text{Req } t \ s \ \ell) \triangleq s \cdot (\#_{\ell} ns \ t) \quad \text{and} \quad \#^{\text{pq}} ns \ pq \triangleq \sum_{r \in pq} \#^{\text{Req}} ns \ r.$$

Lemma

If $\forall r \in pq : r.\text{target} \neq i$, then $\#^{\text{pq}} (i \ t \ e) :: ns \ pq = \#^{\text{pq}} ns \ pq$

Proof.

$\#^{\text{Req}} (i \ t \ e) :: ns \ r = \#^{\text{Req}} ns \ r$ by definition and some case distinction. □

bdd_satcount f vc

Think of a Priority Queue of Req as a *Multiset* with (pure) functions `top()` and `pop()`.

```
1 Req = (target : Uid, sum : ℕ, levels_visited : ℕ)
```

Definition

The priority queue `pq` is *well formed* wrt. list of nodes `ns` if

- $\{r.\text{target} \mid r \in \text{pq}\} \subseteq \{i \mid (i \text{ t } e) \in ns\}$
- $\forall r \in \text{pq} : r.\text{levels_visited} < r.\text{target.level}$

`bdd_satcount f vc`

Think of a Priority Queue of Req as a *Multiset* with (pure) functions `top()` and `pop()`.

```
1 Req = (target : Uid, sum : ℕ, levels_visited : ℕ)
```

Definition

The priority queue `pq` is *well formed* wrt. list of nodes `ns` if

- $\{r.\text{target} \mid r \in \text{pq}\} \subseteq \{i \mid (i \text{ t } e) \in ns\}$
- $\forall r \in \text{pq} : r.\text{levels_visited} < r.\text{target.level}$

Lemma

An empty priority queue is well formed.

Proof.

Trivial. □

bdd_satcount f vc

Think of a Priority Queue of Req as a *Multiset* with (pure) functions `top()` and `pop()`.

```
1 Req = (target : Uid, sum : ℕ, levels_visited : ℕ)
```

Definition

The priority queue `pq` is *well formed* wrt. list of nodes `ns` if

- $\{r.\text{target} \mid r \in \text{pq}\} \subseteq \{i \mid (i \text{ t } e) \in ns\}$
- $\forall r \in \text{pq} : r.\text{levels_visited} < r.\text{target.level}$

Lemma

If `pq` is well formed then `pq.pop()` is too.

Proof.

A little bit of set theory.



bdd_satcount f vc

Accumulate from pq the sum, sacc, and the number of visited levels, lacc, along all in-going edges to a single node with Uid t.

```
1 combine_paths' (pq : PQ<Req>) (tgt : Uid) ((sacc, lacc) :  $\mathbb{N} \times \mathbb{N}$ ) =
2   case pq.top() of
3     None                → (sacc, lacc, pq)
4     Some Req tgt' s  $\ell$  → if tgt'  $\neq$  tgt
5                           then (sacc, lacc, pq)
6                           else let acc' = (sacc. $2^{\ell-lacc}$  + s,  $\ell$ )
7                                ; pq'  = pq.pop()
8                                in combine_paths' pq' tgt acc'

9 combine_paths (pq : PQ<Req>) (tgt : Uid) =
10   combine_paths' pq tgt (0,0)
```

bdd_satcount f vc

Accumulate from pq the sum, sacc, and the number of visited levels, lacc, along all in-going edges to a single node with Uid t.

```
1 combine_paths' (pq : PQ<Req>) (tgt : Uid) ((sacc, lacc) :  $\mathbb{N} \times \mathbb{N}$ ) =  
2   case pq.top() of  
3     None                → (sacc, lacc, pq)  
4     Some Req tgt' s  $\ell$  → if tgt'  $\neq$  tgt  
5                           then (sacc, lacc, pq)  
6                           else let acc' = (sacc. $2^{\ell-lacc}$  + s,  $\ell$ )  
7                               ; pq'  = pq.pop()  
8                               in combine_paths' pq' tgt acc'
```

Lemma

Let $(s', \ell', pq') = \text{combine_paths } pq \ t$, if $\forall r \in pq : r.target \geq i$, then $pq' = \{r \in pq \mid r.target \neq t\}$.

bdd_satcount f vc

Accumulate from pq the sum, sacc, and the number of visited levels, lacc, along all in-going edges to a single node with Uid t.

```
1 combine_paths' (pq : PQ<Req>) (tgt : Uid) ((sacc, lacc) :  $\mathbb{N} \times \mathbb{N}$ ) =  
2   case pq.top() of  
3     None                → (sacc, lacc, pq)  
4     Some Req tgt' s  $\ell$  → if tgt'  $\neq$  tgt  
5                           then (sacc, lacc, pq)  
6                           else let acc' = (sacc. $2^{\ell-lacc}$  + s,  $\ell$ )  
7                               ; pq' = pq.pop()  
8                               in combine_paths' pq' tgt acc'
```

Lemma

Let $(s', \ell', pq') = \text{combine_paths } pq \ t$, if pq is well formed then pq' is too.

bdd_satcount f vc

Accumulate from pq the sum, sacc, and the number of visited levels, lacc, along all in-going edges to a single node with Uid t.

```
1 combine_paths' (pq : PQ<Req>) (tgt : Uid) ((sacc, lacc) : ℕ × ℕ) =
2   case pq.top() of
3     None                → (sacc, lacc, pq)
4     Some Req tgt' s ℓ → if tgt' ≠ tgt
5                          then (sacc, lacc, pq)
6                          else let acc' = (sacc · 2ℓ-lacc + s, ℓ)
7                                ; pq'   = pq.pop()
8                                in combine_paths' pq' tgt acc'
```

Lemma

Let $(s', \ell', pq') = \text{combine_paths } pq \ t$, then, $\#^{pq} \text{ ns } pq = \#^{pq} \text{ ns } pq' + s' \cdot \#_{\ell'} \text{ ns } t$.

bdd_satcount f vc

Forward sum, s , and number of visited levels, ℓ , along an out-going edge to target ptr .

```
1 forward_paths (pq : PQ<Req>) (ptr : Ptr) (s : ℕ) (ℓ : ℕ) =  
2   case s, ptr of  
3     0, _           → (0, pq) (* Well formed ↯ fully connected *)  
4     _, Leaf False → (0, pq)  
5     _, Leaf True  → (s · 2vc-ℓ, pq)  
6     _, Node tgt   → (0, pq + {(tgt, s, ℓ)})
```

bdd_satcount f vc

Forward sum, s , and number of visited levels, ℓ , along an out-going edge to target ptr .

```
1 forward_paths (pq : PQ<Req>) (ptr : Ptr) (s : N) ( $\ell$  : N) =  
2   case s, ptr of  
3     0, _            $\rightarrow$  (0, pq) (* Well formed  $\nrightarrow$  fully connected *)  
4     _, Leaf False  $\rightarrow$  (0, pq)  
5     _, Leaf True   $\rightarrow$  ( $s \cdot 2^{vc-\ell}$ , pq)  
6     _, Node tgt    $\rightarrow$  (0, pq + {(tgt, s,  $\ell$ )})
```

Lemma

Let $(s', pq') = \text{forward_paths } pq \ t \ s \ \ell$. If $\ell < vc$,
then $\#^{pq} \ ns \ pq + s \cdot \#_{\ell} \ ns \ t = \#^{pq'} \ ns \ pq' + s'$.

Proof.

Case analysis and definition of $\#^{pq}$, $\#^{\text{Req}}$, and $\#_{\ell}$. □

bdd_satcount f vc

Forward sum, s , and number of visited levels, ℓ , along an out-going edge to target ptr .

```
1 forward_paths (pq : PQ<Req>) (ptr : Ptr) (s : N) ( $\ell$  : N) =  
2   case s, ptr of  
3     0, _            $\rightarrow$  (0, pq) (* Well formed  $\nrightarrow$  fully connected *)  
4     _, Leaf False  $\rightarrow$  (0, pq)  
5     _, Leaf True   $\rightarrow$  ( $s \cdot 2^{\text{vc} - \ell}$ , pq)  
6     _, Node tgt    $\rightarrow$  (0, pq + {(tgt, s,  $\ell$ )})
```

Lemma

Let $(s', pq') = \text{forward_paths } pq \ t \ s \ \ell$. If $t \in ns$ and pq is well formed, then pq' is also well formed.

Proof.

Case analysis and assumptions.



bdd_satcount f vc

Forward sum, s , and number of visited levels, ℓ , along an out-going edge to target ptr .

```
1 forward_paths (pq : PQ<Req>) (ptr : Ptr) (s : N) ( $\ell$  : N) =  
2   case s, ptr of  
3     0, _            $\rightarrow$  (0, pq) (* Well formed  $\nrightarrow$  fully connected *)  
4     _, Leaf False  $\rightarrow$  (0, pq)  
5     _, Leaf True   $\rightarrow$  ( $s \cdot 2^{\text{vc} - \ell}$ , pq)  
6     _, Node tgt    $\rightarrow$  (0, pq + {(tgt, s,  $\ell$ )})
```

Lemma

Let $(s', pq') = \text{forward_paths } pq \ t \ s \ \ell$. If $t = \text{Leaf } _$, then $pq' \subseteq pq$.

If $t = \text{Node } u$, then $pq' \subseteq pq + \{(\text{Req } u \ s \ \ell)\}$.

Proof.

Case analysis and assumptions.



bdd_satcount f vc

Accumulate all in-going edges and then forward to children (to-be processed later).

```
1 bdd_satcount' (ns : List<Node>) (pq : PQ<Req>) (racc : ℕ) =
2   case ns      , pq.top() of
3     _          , None   → racc
4     n::ns', _        →
5       let (s, ℓ, pq') = combine_paths pq    n.i
6         ; (rt, pq'') = forward_paths pq'  n.t s (ℓ+1)
7         ; (re, pq''') = forward_paths pq'' n.e s (ℓ+1)
8       in bdd_satcount' ns' pq''' (racc + rt + re)
```

`bdd_satcount f vc`

Accumulate all in-going edges and then forward to children (to-be processed later).

```
1 bdd_satcount' (ns : List<Node>) (pq : PQ<Req>) (racc : ℕ) = ...
```

Lemma

Assume pq and ns are well formed and $\{n.uid.level \mid n \in ns\} \subseteq \{0, 1, \dots, vc - 1\}$.

Then, $bdd_satcount' ns pq r = r + \#^{pq} ns pq$.

Proof.

Induction in ns and case analysis on top of pq . Use previous lemmata to skip node (if not the target) or to parse correctness through `combine_paths` and `forward_paths`.

To this end, one needs to bound the number of visited levels in each request by vc . Furthermore, the results, $racc$, rt , and re , are combined with the lemmata for $\#_n$ (Appendix). □

bdd_satcount f vc

Finally, deal with the root for a BDD f .

```
1 bdd_satcount (False : Bdd.Leaf) (vc : ℕ) = 0
2 bdd_satcount (True  : Bdd.Leaf) (vc : ℕ) = 2vc
3 bdd_satcount (r::ns : Bdd.Nodes) (vc : ℕ) =
4   let pq      = ∅
5   ; (rt, pq') = forward_paths pq  r.t 1 1
6   ; (re, pq'') = forward_paths pq' r.e 1 1
7   in bdd_satcount' ns ∅ (rt + re)
```

`bdd_satcount f vc`

Finally, deal with the root for a BDD f .

```
1 bdd_satcount ( _ : Bdd) (vc : ℕ) = ...
```

Theorem

If f is well formed (incl. $\{n.uid.level \mid n \in Bdd.Nodes\ ns\} \subseteq \{0, 1, \dots, vc - 1\}$), then $bdd_satcount\ f\ vc = \#_0 f$.

Proof.

Leaf cases are trivial. For nodes ns , use lemmata for `forward_paths` to prove preconditions for `bdd_satcount' ns pq (0,0)` correctness. □

Take Home Message...

- Time-forward processing algorithms can be implemented *functionally*.
 - They are *pure* and *tail-recursive*.
 - They are I/O-efficient since they only work on lists and trees [Blelloch & Harper].
- Proving correctness is feasible (see also github.com/SSoelvsten/cadiar)
 - Further refinement possible to get closer to the C++ performance.
- One can prove them to be efficient both with respect to time and I/O complexity.

Steffan Christ Sølvsten

✉ soelvsten@cs.au.dk

🌐 ssoelvsten.github.io

Adiar

🔗 github.com/ssoelvsten/adiar

📖 ssoelvsten.github.io/adiar

Contents

Motivation

Correctness of Time-forward Processing

Encoding Binary Decision Diagrams

`bbd_eval`

`bbd_not`

`bbd_satcount`

Appendix

$[a]_n$: Bounded Domain

$\#_n$: Number of Assignments

Bounded Domain

For this, we need to work with Boolean functions with a bounded domain.

Let the *variable count*, vc , be fixed.

Definition

$$\lfloor a \rfloor_n \triangleq (a \overline{\{i \mid n \leq i < vc\}} \subseteq \{\perp\}).$$

Bounded Domain

For this, we need to work with Boolean functions with a bounded domain.
Let the *variable count*, vc , be fixed.

Definition

$$\lfloor a \rfloor_n \triangleq (a \overline{\{i \mid n \leq i < vc\}} \subseteq \{\perp\}).$$

Lemma (alternative definition)

$$\lfloor a \rfloor_n \iff \forall i : i \notin \{n, n+1, \dots, vc-1\} \implies a \ i = \perp.$$

Proof.

From definition.



Bounded Domain

For this, we need to work with Boolean functions with a bounded domain.
Let the *variable count*, vc , be fixed.

Definition

$$\lfloor a \rfloor_n \triangleq (a \overline{\{i \mid n \leq i < vc\}} \subseteq \{\perp\}).$$

Lemma

If $n > vc$, then $\lfloor a \rfloor_n \iff a = \lambda_.\perp$.

Proof.

From (alternative) definition. □

Bounded Domain

For this, we need to work with Boolean functions with a bounded domain.
Let the *variable count*, vc , be fixed.

Definition

$$\lfloor a \rfloor_n \triangleq \overline{(a \{i \mid n \leq i < vc\} \subseteq \{\perp\})}.$$

Lemma

For all $n \in N$, $\lfloor a \rfloor_n \wedge \neg a \ n \iff \lfloor a \rfloor_{n+1}$.

Proof.

Case analysis of $i = n$ and (alternative) definition. □

Bounded Domain

For this, we need to work with Boolean functions with a bounded domain.

Let the *variable count*, vc , be fixed.

Definition

$$\lfloor a \rfloor_n \triangleq \overline{(a \{i \mid n \leq i < vc\} \subseteq \{\perp\})}.$$

Lemma

For all $n \in N$, $\lfloor a \rfloor_n \iff (\lfloor a \rfloor_n \wedge a_n) \vee \lfloor a \rfloor_{n+1}$.

Proof.

Case analysis of $i = n$ and (alternative) definition. □

Bounded Domain

For this, we need to work with Boolean functions with a bounded domain.

Let the *variable count*, vc , be fixed.

Definition

$$\lfloor a \rfloor_n \triangleq \overline{(a \{i \mid n \leq i < vc\} \subseteq \{\perp\})}.$$

Lemma

$\{a \mid \lfloor a \rfloor_n\}$ is finite.

Proof.

Induction in n and some set theory.



Bounded Domain

For this, we need to work with Boolean functions with a bounded domain.
Let the *variable count*, vc , be fixed.

Definition

$$\lfloor a \rfloor_n \triangleq \overline{(a \mid \{i \mid n \leq i < vc\} \subseteq \{\perp\})}.$$

Lemma

If $n < vc$, $|\{a \mid \lfloor a \rfloor_n \wedge a \ n\}| = |\{a \mid \lfloor a \rfloor_{n+1}\}|$.

Proof.

Previous lemmas together with set theory.



Bounded Domain

For this, we need to work with Boolean functions with a bounded domain.

Let the *variable count*, vc , be fixed.

Definition

$$\lfloor a \rfloor_n \triangleq (a \overline{\{i \mid n \leq i < vc\}} \subseteq \{\perp\}).$$

Lemma

If $n < vc$, $|\{a \mid \lfloor a \rfloor_{vc-n}\}| = 2^n$ and $|\{a \mid \lfloor a \rfloor_n\}| = 2^{vc-n}$.

Proof.

Induction in n , case analysis on dom_bounded with a and with $vc - n$ and $vc - (n + 1)$, respectively, and some set theory. □

Bounded Domain

For this, we need to work with Boolean functions with a bounded domain.

Let the *variable count*, vc , be fixed.

Definition

$$\lfloor a \rfloor_n \triangleq (a \overline{\{i \mid n \leq i < vc\}} \subseteq \{\perp\}).$$

Lemma

If $\lfloor a \rfloor_n$, then $\lfloor (a[x := \perp]) \rfloor_n$.

Proof.

From (alternative) definition. □

Bounded Domain

For this, we need to work with Boolean functions with a bounded domain.

Let the *variable count*, vc , be fixed.

Definition

$$\lfloor a \rfloor_n \triangleq \overline{(a \{i \mid n \leq i < vc\} \subseteq \{\perp\})}.$$

Lemma

If $n \leq x \leq vc$, then $\lfloor a \rfloor_n \iff \lfloor a[x := \mathbb{B}] \rfloor_n$.

Proof.

From (alternative) definition. □

Bounded Domain

For this, we need to work with Boolean functions with a bounded domain.

Let the *variable count*, vc , be fixed.

Definition

$$\lfloor a \rfloor_n \triangleq (a \ \overline{\{i \mid n \leq i < vc\}} \subseteq \{\perp\}).$$

Lemma

If $x < t.level$, then $bdd_eval' \ ns \ t \ a[x := \mathbb{B}] = bdd_eval' \ ns \ t \ a$.

Proof.

Induction in ns with a case analysis of the algorithms branches. Here, use that the BDD is *well formed*. □

Bounded Domain

For this, we need to work with Boolean functions with a bounded domain.

Let the *variable count*, vc , be fixed.

Definition

$$\lfloor a \rfloor_n \triangleq \overline{(a \mid \{i \mid n \leq i < vc\} \subseteq \{\perp\})}.$$

Lemma

$$|\{a \mid \neg a \times \wedge bdd_eval' \ ns \ t \ a \wedge \lfloor a \rfloor_n\}| = |\{a \mid bdd_eval' \ ns \ t \ a \wedge \lfloor a \rfloor_{n+1}\}|.$$

Proof.

Set theory and previous lemma $bdd_eval' \ ns \ t \ (a[x := \mathbb{B}])$. □

Number of Assignments

We need a mathematical notion of the number of assignments satisfying a BDD.

Definition

$$\#_n f \triangleq |\{a \mid \text{bdd_eval } f \ a \wedge \lfloor a \rfloor_n\}|.$$

Number of Assignments

We need a mathematical notion of the number of assignments satisfying a BDD.

Definition

$$\#_n f \triangleq |\{a \mid \text{bdd_eval } f \ a \wedge \lfloor a \rfloor_n\}|.$$

Lemma

If $n \leq vc$, $\#_n \top = 2^{vc-n}$.

Proof.

From definition of `bdd_eval` and lemma on $\lfloor a \rfloor_n$.



Number of Assignments

We need a mathematical notion of the number of assignments satisfying a BDD.

Definition

$$\#_n f \triangleq |\{a \mid \text{bdd_eval } f \ a \wedge \lfloor a \rfloor_n\}|.$$

Lemma

$$\#_n \perp = 0.$$

Proof.

From definition of `bdd_eval`.



Number of Assignments

We need a mathematical notion of the number of assignments satisfying a BDD.

Definition

$$\#_n f \triangleq |\{a \mid \text{bdd_eval } f \ a \wedge \lfloor a \rfloor_n\}|.$$

Lemma

$$\#_n f = 2^{n-|\text{support}(f)|} \cdot \#_{\text{support}(f)} f.$$

Number of Assignments

We need a mathematical notion of the number of assignments satisfying a BDD.

Definition

$$\#_n f \triangleq |\{a \mid \text{bdd_eval } f \ a \wedge \lfloor a \rfloor_n\}|.$$

Lemma

$$\#_n \text{Node}(i \ t \ e) :: ns = \#_{n+1}^t ns + \#_{n+1}^{\neg t} ns$$

Proof.

Use *well formedness*, *set theory*, and previous lemmata about $\lfloor a \rfloor_n$ to prove:

- $a \ i \implies \text{bdd_eval}' \ ns \ t \ a \wedge \lfloor a \rfloor_n$ and $\neg a \ i \implies \text{bdd_eval}' \ ns \ e \ a \wedge \lfloor a \rfloor_n$.
- Can split set of assignments S into $S_t \cup S_e$.
- $S_t = \{a \mid a \ x \wedge \text{bdd_eval}' \ ns \ e \ a \wedge \lfloor a \rfloor_{n+1}\}$ and $S_e = \{a \mid \neg a \ x \wedge \text{bdd_eval}' \ ns \ t \ a \wedge \lfloor a \rfloor_{n+1}\}$.
- $S_t \cap S_e = \emptyset$ and hence $|S| = |S_t| + |S_e|$.



Number of Assignments

We need a mathematical notion of the number of assignments satisfying a BDD.

Definition

$$\#_n f \triangleq |\{a \mid \text{bdd_eval } f \ a \wedge \lfloor a \rfloor_n\}|.$$

Lemma

If $n.\text{uid} < t$, then $\text{bdd_eval}' \ n :: \text{ns } t \ a = \text{bdd_eval}' \ \text{ns } t \ a$.

Proof.

Simple case analysis.



Number of Assignments

We need a mathematical notion of the number of assignments satisfying a BDD.

Definition

$$\#_n f \triangleq |\{a \mid \text{bdd_eval } f \ a \wedge \lfloor a \rfloor_n\}|.$$

Lemma

$$\#_n (i \ t \ e) :: ns \ t = \#_n \ ns \ t \text{ and } \#_n \ n :: ns \ e = \#_n \ ns \ e.$$

Proof.

Due to levels are *strictly* increasing in BDDs.



Number of Assignments

We need a mathematical notion of the number of assignments satisfying a BDD.

Definition

$$\#_n f \triangleq |\{a \mid \text{bdd_eval } f \ a \wedge \lfloor a \rfloor_n\}|.$$

Lemma

If $i \neq u$, then $\#_n (i \text{ t } e) :: ns \ u = \#_n ns \ u$.

Proof.

By definition.

